55

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

GitHub Actions Temelleri: Workflows, Jobs, Steps ve Runners

GitHub Actions Fundamentals: Workflows, Jobs, Steps, and Runners

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kodunuzu GitHub'a `git push` yaptığınız anda harekete geçen görünmez bir fabrika düşünün! 1) Workflow: Fabrikanın üretim planıdır (`.github/workflows/ci.yaml`). 2) Trigger (Tetikleyici): Zilin çalmasıdır ('Main branch'e kod geldi!'). 3) Jobs (İşler): Fabrikanın farklı departmanlarıdır ('Biri test yapsın, diğeri Docker derlesin'). 4) Runners: Bu işleri yapan bulut bilgisayarlarıdır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Imagine an invisible, robotic smart factory that fires up the millisecond you run `git push`! 1) Workflow: The factory blueprint (`.github/workflows/ci.yaml`). 2) Trigger (Event): The starting bell ringing ('Code pushed to main!'). 3) Jobs: Distinct assembly lines ('Line 1 tests Java, Line 2 bakes the container'). 4) Runners: The virtual machines in the cloud executing the steps!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

GitHub Actions mimarisinin yapı taşları: 1) `Events`: İş akışını başlatan olaylar (`push`, `pull_request`, `schedule`, `workflow_dispatch`). 2) `Jobs`: Varsayılan olarak paralel çalışan bağımsız görev bloklarıdır (`needs: [build]` ile sıralı bağımlılık kurulur). 3) `Steps`: Sırayla çalıştırılan tekil kabuk komutları (`run`) veya hazır topluluk eylemleri (`uses: actions/checkout@v4`). 4) `Runners`: Ubuntu, macOS veya Windows barındıran geçici sanal makinelerdir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

The core taxonomy of GitHub Actions: 1) `Events` trigger execution (`push`, `pull_request`, webhooks). 2) `Jobs` run concurrently by default on isolated compute runners unless sequenced via `needs: [jobA]`. 3) `Steps` execute sequentially inside a job environment as shell scripts (`run`) or composite action modules (`uses`). 4) `Runners` are ephemeral hosted virtual machines or self-hosted servers.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# .github/workflows/hello-ci.yaml
name: First Spring Boot Workflow
on:
  push:
    branches: [ "main" ]

jobs:
  say-hello:
    runs-on: ubuntu-latest
    steps:
      - name: Kodu Klonla
        uses: actions/checkout@v4

      - name: Terminale Yazdır
        run: |
          echo "🚀 Java Spring Boot CI Pipeline Başlatıldı!"
          echo "Commit SHA: ${{ github.sha }}"
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Bir iş akışındaki adımlar (steps) aynı sanal makinede ardışık çalışır ve dosya sistemini paylaşır. Ancak farklı 'Job'lar farklı sanal makinelerde çalıştığı için aralarında dosya paylaşımı yapmak istiyorsanız `actions/upload-artifact` kullanmalısınız!

🇬🇧 WATCH OUT

Steps within a job share the runner filesystem; distinct Jobs run on separate virtual machines! Pass compiled artifacts between jobs using `upload-artifact` and `download-artifact`.

🎯 Özet Çıkarımlar

- GitHub Actions harici CI sunucusu (Jenkins) kurma ihtiyacını ortadan kaldırır.
- YAML tabanlı bildirici sözdizimi doğrudan projenin Git reposunda saklanır.
- Geniş GitHub Marketplace ekosistemi hazır otomasyon eylemleri sunar.

🎯 Key Takeaways

- Eliminates the maintenance overhead of managing dedicated external CI/CD servers.
- Declarative workflow definitions live directly inside your project repository.
- Leverages a vast marketplace of battle-tested open-source actions for rapid setup.